

Permissões de arquivos no Linux

Descrição do projeto

Este projeto demonstra como realizar alguns comandos no Linux para quem trabalha na área de segurança cibernética e faz o controle de permissões de arquivos e diretórios para usuários e grupos de usuários. Realizei os comandos pelo Shell Bash no laboratório do curso: Tools of the Trade: Linux and SQL da instituição Coursera.

Verificar detalhes de arquivos e diretórios

O comando ls serve para exibir os nomes dos arquivos e diretórios no diretório de trabalho atual. Podemos adicionar complementos a este comando, por exemplo o ls -a mostra os arquivos ocultos existentes naquele diretório, e o ls -l mostrará as permissões definidas para cada arquivo e diretório, incluindo a data da última alteração e entre outras informações. Combinando esses dois comandos, podemos usar o ls -la para mostrar as permissões de cada arquivo e diretório, incluindo os que estão marcados como ocultos.

```
researcher2@d8e0ef76259e:~$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep  3 20:22 .
drwxr-xr-x 1 root          root          4096 Sep  3 19:45 ..
-rw----- 1 researcher2 research_team  382 Sep  3 21:13 .bash_history
-rw-r--r-- 1 researcher2 research_team  220 Mar 27  2022 .bash_logout
-rw-r--r-- 1 researcher2 research_team 3574 Sep  3 19:45 .bashrc
-rwxrwxrwx 1 researcher2 research_team 3574 Sep  3 19:45 .profile
drw-r--r-- 3 researcher2 research_team 4096 Sep  3 19:45 projects
researcher2@d8e0ef76259e:~$
```

Descrever a string de permissões

Agora que já rodamos o comando para descobrir quais são as permissões de cada arquivo e diretório dentro do diretório atual, vamos explorar o que significa cada caractere na string de permissões utilizando o diretório projects como exemplo. Veja que a primeira informação que aparece em cada linha é uma sequência de 10 caracteres. O diretório projects aparece neste modelo: "drwxr-xr-x". O "d" representa que se trata de um diretório, caso fosse um arquivo, começaria com o caractere "-". O segundo, terceiro e quarto caracteres representam as permissões do proprietário "usuário = u", e aqui neste diretório, ele possui permissão para "ler = r", "gravar = w", e "executar = x" arquivos e programas no diretório. O quinto, sexto e sétimo caractere representam as permissões do "grupo = g" onde o usuário está inserido, e os 3 caracteres finais representam as permissões de "outros = o", que representa qualquer outro usuário do sistema. Note que, nos seguintes grupos, alguns caracteres de permissão foram

substituídos por “-”, o que significa que aquele grupo de proprietários não possui aquele tipo de permissão.

```
researcher2@d8e0ef76259e:~$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep  3 20:22 .
drwxr-xr-x 1 root          root          4096 Sep  3 19:45 ..
-rw----- 1 researcher2 research_team    6 Sep  3 20:22 .bash_history
-rw-r--r-- 1 researcher2 research_team  220 Mar 27 2022 .bash_logout
-rw-r--r-- 1 researcher2 research_team 3574 Sep  3 19:45 .bashrc
-rw-r--r-- 1 researcher2 research_team 3574 Sep  3 19:45 .profile
drwxr-xr-x 3 researcher2 research_team 4096 Sep  3 19:45 projects
researcher2@d8e0ef76259e:~$
```

Alterar permissões de arquivos

Para alterar as permissões de um determinado tipo de proprietário, utilizamos o comando `chmod`, um operador matemático, sendo que “-” é usado para remover uma permissão, “+” para adicionar e “=” para sobrescrever e o caractere que representa a permissão. Neste exemplo, estamos no diretório `projects` e utilizei o comando `ls -l` para exibir todos os arquivos e diretórios visíveis no diretório atual. Verificando as permissões do arquivo `project_k.txt`, percebemos que todos os proprietários possuem permissão para leitura e gravação. Utilizei o comando `chmod` para remover a permissão de gravação para todos os grupos de proprietário, e utilizando o comando `ls -l`, vemos que as permissões realmente foram removidas.

```
researcher2@d8e0ef76259e:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Sep  3 19:45 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Sep  3 19:45 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Sep  3 19:45 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep  3 19:45 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep  3 19:45 project_t.txt
researcher2@d8e0ef76259e:~/projects$ chmod u-w,g-w,o-w project_k.txt
researcher2@d8e0ef76259e:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Sep  3 19:45 drafts
-r--r--r-- 1 researcher2 research_team  46 Sep  3 19:45 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Sep  3 19:45 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep  3 19:45 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep  3 19:45 project_t.txt
researcher2@d8e0ef76259e:~/projects$
```

Alterar permissões de arquivos ocultos

Agora, para alterar as permissões de um arquivo oculto, utilizamos o mesmo método, porém precisamos especificar que se trata de um arquivo oculto. Note que utilizei o comando `ls -la` para verificar os detalhes de arquivos e diretórios visíveis e ocultos. Neste exemplo, utilizei o

chmod para adicionar permissões de execução para o proprietário usuário, e adicionei permissões de escrita para o grupo e o proprietário outros. Para especificar que se trata de um arquivo oculto, adicionamos o caractere “.” antes do nome do arquivo, exatamente como ele aparece no Shell Bash quando o consultamos. Utilizando o comando ls -la novamente, verificamos que as permissões foram adicionadas.

```
researcher2@d8e0ef76259e:~$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep  3 20:22 .
drwxr-xr-x 1 root          root        4096 Sep  3 19:45 ..
-rw----- 1 researcher2 research_team   46 Sep  3 20:42 .bash_history
-rw-r--r-- 1 researcher2 research_team  220 Mar 27  2022 .bash_logout
-rw-r--r-- 1 researcher2 research_team 3574 Sep  3 19:45 .bashrc
-rw-r--r-- 1 researcher2 research_team 3574 Sep  3 19:45 .profile
drw-r--r-- 3 researcher2 research_team 4096 Sep  3 19:45 projects
researcher2@d8e0ef76259e:~$ chmod u+x,g+w,o+w .profile
researcher2@d8e0ef76259e:~$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep  3 20:22 .
drwxr-xr-x 1 root          root        4096 Sep  3 19:45 ..
-rw----- 1 researcher2 research_team   80 Sep  3 20:49 .bash_history
-rw-r--r-- 1 researcher2 research_team  220 Mar 27  2022 .bash_logout
-rw-r--r-- 1 researcher2 research_team 3574 Sep  3 19:45 .bashrc
-rwxrw-rw- 1 researcher2 research_team 3574 Sep  3 19:45 .profile
drw-r--r-- 3 researcher2 research_team 4096 Sep  3 19:45 projects
researcher2@d8e0ef76259e:~$
```

Alterar permissões de diretórios

Neste exemplo, utilizei o chmod para remover a permissão de execução dos proprietários usuário = u, grupo = g e outros = o, do diretório projects, e após a alteração, utilizei o comando ls -l para consultar as permissões do diretório e verificar se as alterações foram realmente aplicadas. Note que para todos os proprietários, o caractere “x” foi substituído por “-”, o que significa que a permissão já foi removida.

```
researcher2@d8e0ef76259e:~$ ls -l
total 4
drwxr-xr-x 3 researcher2 research_team 4096 Sep  3 19:45 projects
researcher2@d8e0ef76259e:~$ chmod u-x,g-x,o-x projects
researcher2@d8e0ef76259e:~$ ls -l
total 4
drw-r--r-- 3 researcher2 research_team 4096 Sep  3 19:45 projects
researcher2@d8e0ef76259e:~$
```

Resumo

Neste projeto foram explorados comandos do Linux utilizados para visualizar e administrar permissões de arquivos e diretórios. O controle adequado dessas permissões é importante para aplicar o princípio do menor privilégio e reduzir o acesso não autorizado a recursos do sistema.